

Практическая работа 7

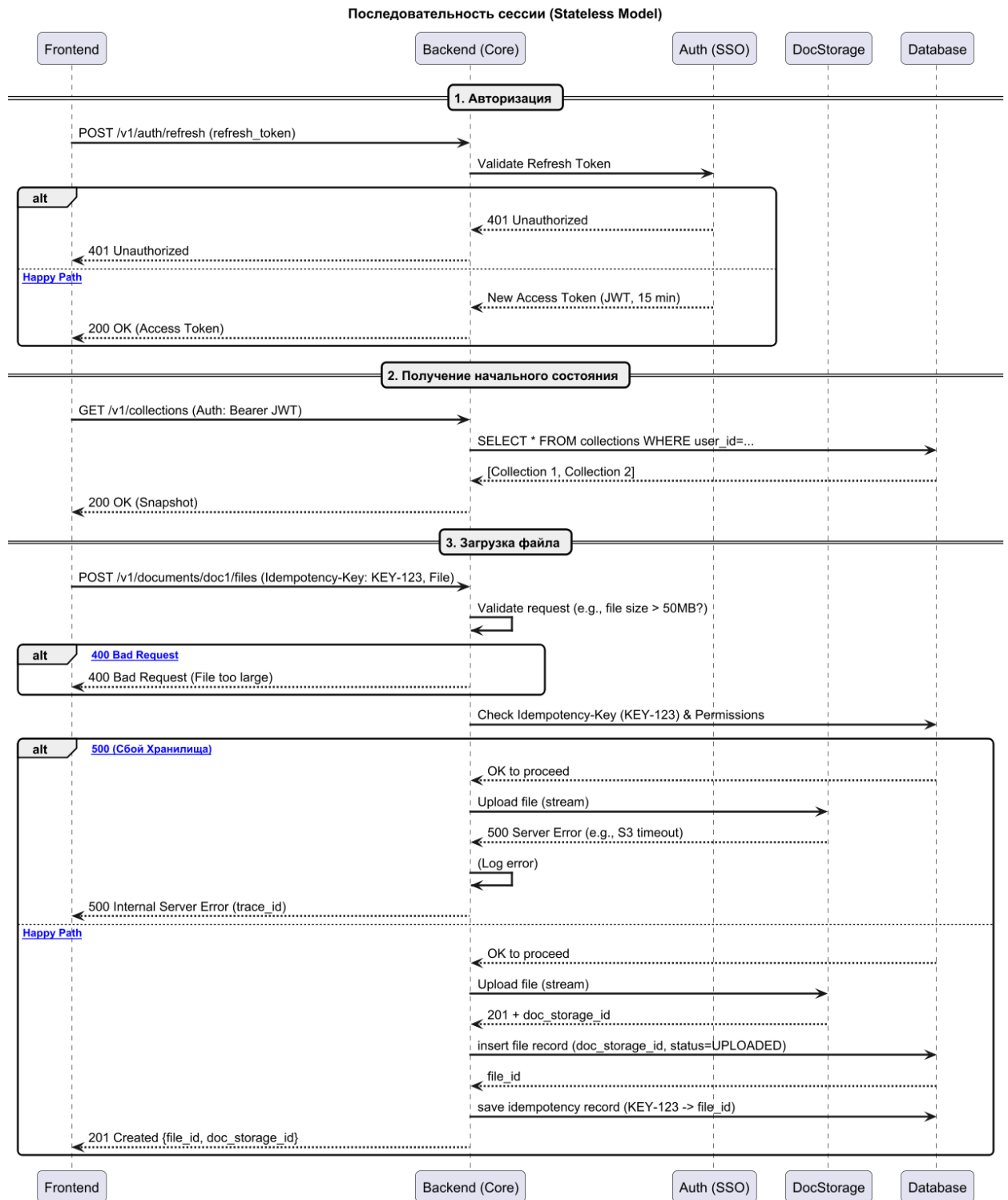
1. Сценарий и Допущения

Сквозной сценарий (поток ценности): «Аутентификация (получение JWT) → Просмотр списка требуемых сборов документов (Join/Snapshot) → Загрузка файла (обмен состоянием, POST) → Подтверждение отправки на верификацию (POST /submit) → Выход (Logout или истечение токена)».

Допущения:

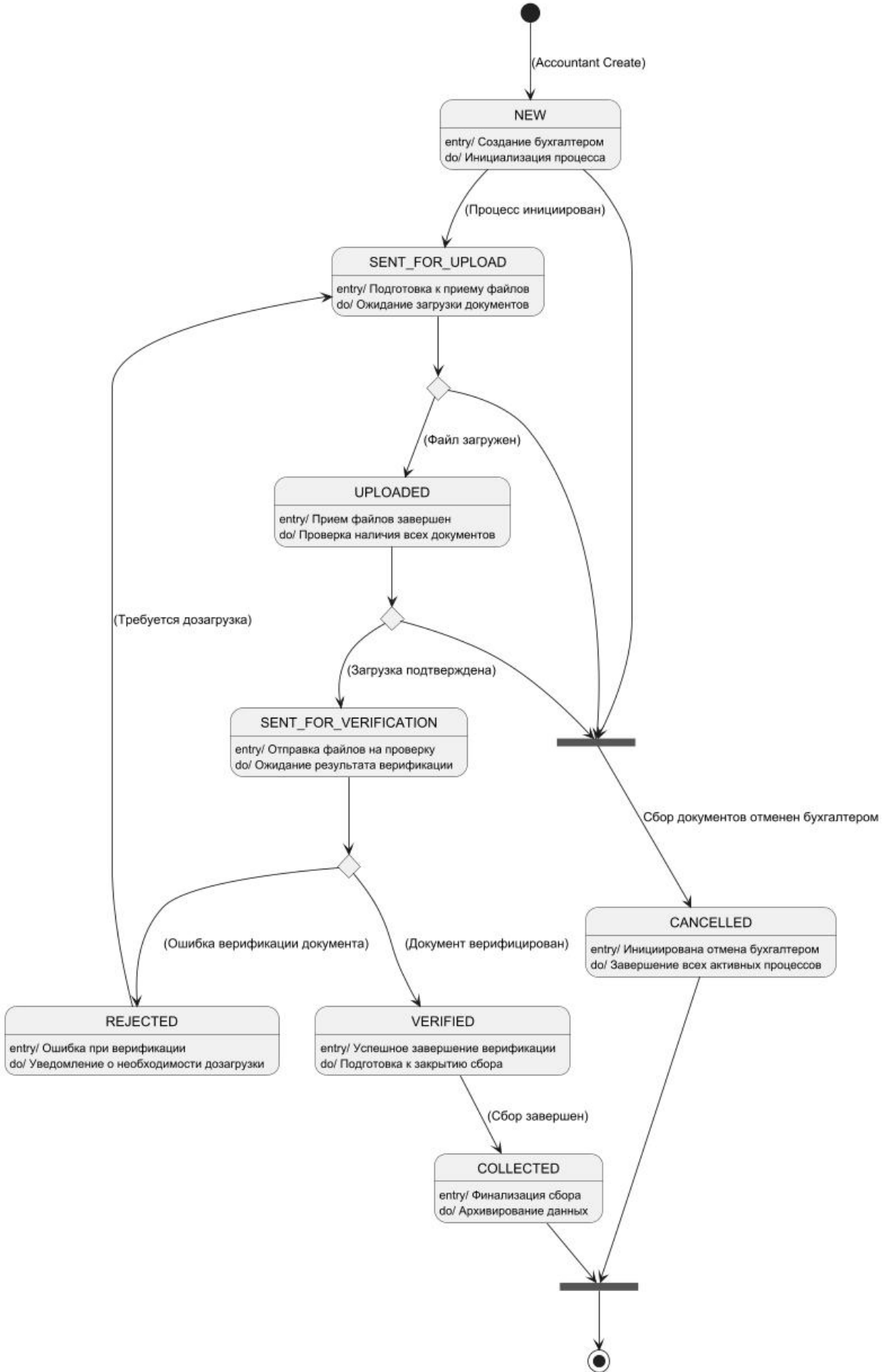
- **Тип клиента:** Web-приложение
- **Сеть:** WiFi.
- **Целевой регион / RTT:** РФ. Ожидаемый RTT p95 $\approx$ 150 мс.
- **Допустимые потери:** 0.
- **Лимит участников:** 1:1 (Клиент ↔ Сервер).

2. Sequence-диаграмма

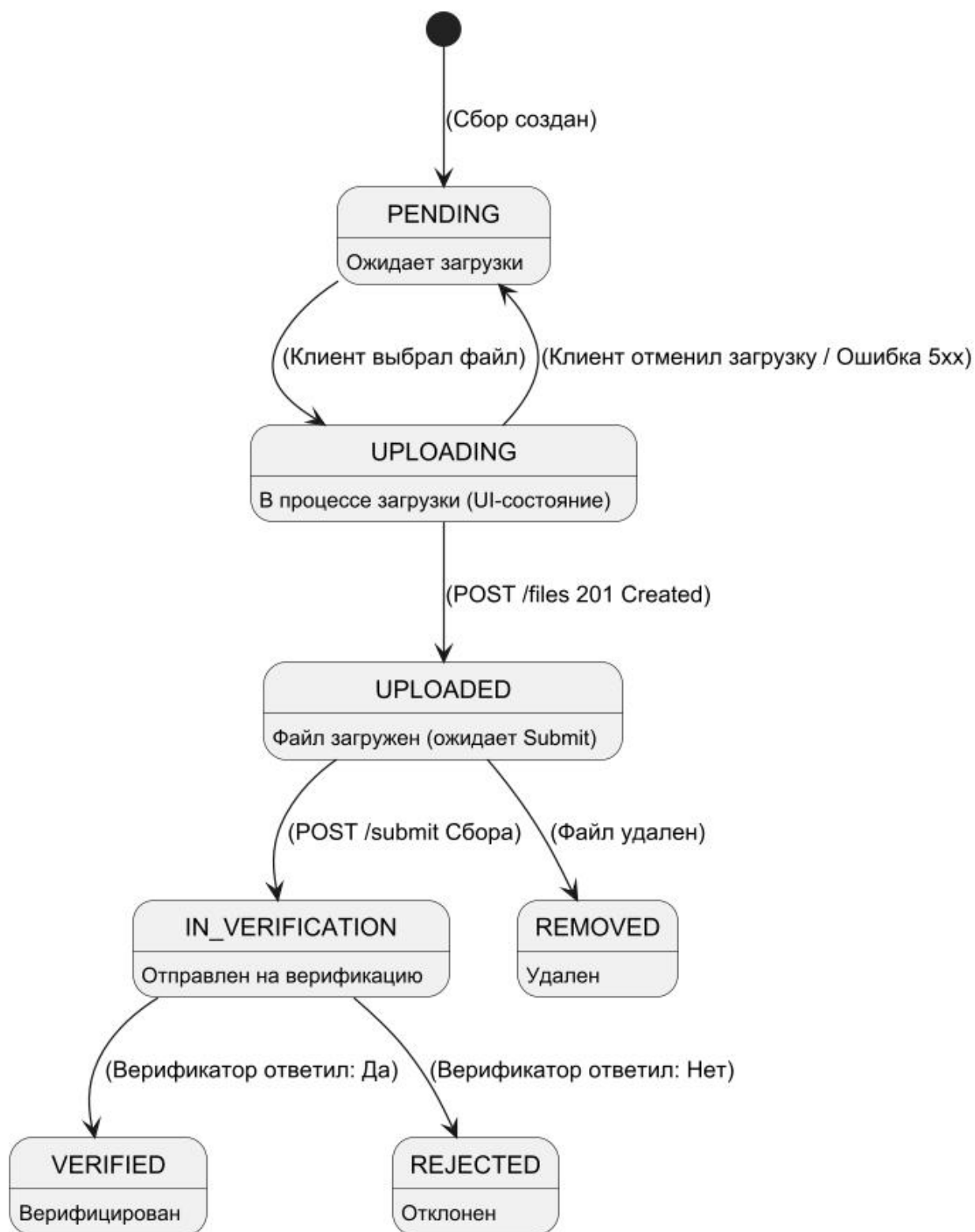


3. State-диаграммы

Состояния Сбора Документов (Collection)



Состояния Запрашиваемого Документа (CollectionDocument)



4. Модель репликации состояния

Используется гибридный подход:

1. Снапшот (Snapshot): При входе в приложение (Join) клиент получает полный снимок состояния (GET /v1/collections) — список всех назначенных ему сборов и их текущие статусы.

2. Команды (Commands): Клиент отправляет на сервер *команды*, которые меняют состояние (POST /v1/documents/{id}/files или POST /v1/collections/{id}/submit).

3. Дельты (Polling): Клиент не использует потоковую репликацию (WebSockets). Для экономии батареи, он периодически (например, каждые 30 секунд или при Pull-to-Restore) опрашивает (GET /v1/collections/{id}) сервер для получения "дельт" — изменившихся статусов (например, status: IN_VERIFICATION -> status: VERIFIED).

Все команды, снапшоты и поллинг идут по HTTPS (TCP). Потеря команды (POST /submit) недопустима.

Матрица «Сущность → Канал → Политика»

Сущность (Что реплицируется)	Канал	Политика (Частота, Надёжность)
Команда: Загрузка Файла	Надёжный (HTTPS POST)	По событию (действие клиента). Ретрай: Да, с Idempotency-Key.
Команда: Submit	Надёжный (HTTPS POST)	По событию (действие клиента). Ретрай: Да, с Idempotency-Key.
Состояние: Статусы сборов	Надёжный (HTTPS GET)	Pull (Polling). Частота: 1 раз в 30 сек (когда приложение активно) + Pull-to-Restore.

5. Окна лаг-компенсации и сглаживания

В нашей ИС нет "лаг-компенсации" в игровом смысле (предсказание движения). Вместо этого мы используем предсказание UI для сглаживания сетевых задержек (RTT 150ms).

- Буфер интерполяции (UI Smoothing):

Сценарий: Клиент нажимает "Загрузить файл".

Предсказание: UI немедленно (0 мс) переводит этот документ в состояние "Загружается..." (локальное предсказание) и показывает спиннер.

Реальность: Запрос уходит на сервер (RTT 150мс), сервер обрабатывает (NFR p95 до 5 сек).

- Окно реконсиляции (Reconciliation):

Это окно — 1 последняя команда ("Upload").

Если (через 5 сек) приходит ответ 201 Created (Authoritative state): UI меняет "Загружается..." на "Загружено" (сверка прошла успешно).

Если (через 5 сек) приходит ответ 500 Server Error: UI меняет "Загружается..." на "Ошибка загрузки" (сверка провалилась) и разблокирует кнопку "Повторить".

6. Таблица параметров обновления

Категория	Параметр	Значение	Обоснование
Частоты	Частота снапшотов (Join)	1 раз / сессию	При входе получаем полный список сборов.
	Частота дельт (Polling)	0.033 Гц (1 раз / 30 сек)	Компромисс между актуальностью статусов и экономией батареи мобильного клиента.
Сеть	Heartbeat (мс)	N/A	Stateless-модель
	Timeout-порог (мс)	15 минут	JWT Access Token TTL. Это "таймаут" сессии.
	Max Packet (Upload)	50 МБ	NFR
Надёжность	Политика ретраев	Idempotency-Key (для POST) / Exponential Backoff (для GET)	
	Max Retries (для GET)	3	
Качество (SLI)	p95 Latency (API GET)	≤ 1 000 мс	NFR

	p95 Latency (API POST File)	$\leq 5\,000$ мс	NFR
	Допустимая потеря (пакетов)	0%	Допущение

7. Связь с профилем

Надёжность: Все данные (файлы, статусы) критически надёжны. Нет "ненадёжных" (UDP) каналов, как в играх. Все идет по HTTPS (TCP).

Асинхронность: "Реальное время" достигается асинхронно. Когда клиент отправляет команду POST /submit, Backend (Core) немедленно отвечает 202 Accepted, а *внутри* бэкенда эта команда попадает в Брокер (BPM, RabbitMQ, как в Практике 1). Клиент *не* ждет верификации, он узнает о ее результате через поллинг.

Stateless: Отсутствие Heartbeat и использование JWT-токенов — ключевое решение для масштабируемости и экономии батареи мобильного клиента.

8. Наблюдаемость

Как мы поймем, что наша *stateless*-модель работает:

Дашборды

1. Auth Health: График 401 Unauthorized. Это наш главный индикатор "проблем с сессией". Если 401 много, значит у клиентов проблемы с обновлением (Refresh) токенов.

2. API Reliability:

- Счетчики (Rate) ошибок 5xx (Сбои Core, DocStorage).
- Счетчики (Rate) ошибок 400 (Клиент шлет невалидные данные).
- Счетчики (Rate) ошибок 409 Conflict (Ошибка идемпотентности).

3. Idempotency: % запросов POST, которые были повторными (показатель того, как часто ретрай в 5% сети спасают клиента).

Тревоги (Alerts)

- alert: High_401_Rate (Если 401 > 5% от запросов) → Проблема с SSO или атака. (Замена "Heartbeat_Drop").

- alert: High_5xx_Rate (Если p99 Latency POST /files > 5 сек) → Деградация DocStorage.

Тестовые сценарии (на бумаге)

1. Сценарий: "Потеря Wifi" (Loss 100% на 20 сек)

- *Действие:* Клиент в приложении. Проблемы на стороне провайдера.
- *Ожидание:* Ничего не происходит. "Сессия" не теряется, так как ее нет на сервере. Следующий API-запрос (e.g., Pull-to-Refresh) просто работает, так как JWT-токен (15 мин) еще валиден.

2. Сценарий: "Двойной Submit" (Retry)

- *Действие:* Клиент жмет "Submit" (POST ... Idempotency-Key: KEY-SUBMIT). Запрос уходит, но ответ от сервера теряется (5% loss).
- *Ожидание:* Клиентский SDK (не видя 202 Accepted) выполняет Retry с тем же Id...Key: KEY-SUBMIT.
- *SLO:* Сервер видит ключ, *не* запускает вторую верификацию, а возвращает кешированный ответ 202 Accepted (как на Sequence-диаграмме). Модель устойчива.

3. Сценарий: "Протухший токен"

- *Действие:* Клиент открыл приложение спустя 1 час (JWT 15 мин истек).
- *Ожидание:* Первый запрос (e.g., GET /collections) возвращает 401. Клиентский SDK *автоматически* использует Refresh Token, получает новый JWT и *прозрачно* повторяет GET /collections. Пользователь видит только чуть более долгую загрузку (1 RTT + RTT Auth).